

INSTITUTO FEDERAL

Mato Grosso do Sul

Prog. Disp. Móveis II

Milena Alegre

Localização Em Tempo Real

Dispositivos Físicos, Permissões Nativas e Implementação no React Native.

Sobre o funcionamento do GPS, arquitetura de permissões nativas (iOS/Android).

Emuladores não possuem antenas de GPS reais. Eles apenas simulam coordenadas estáticas.

Para testar bússola, giroscópio e precisão em movimento, o hardware real é obrigatório.

Precisamos validar na prática como o iOS e o Android bloqueiam ou permitem o acesso aos sensores.

Multiplas Fontes de Dados

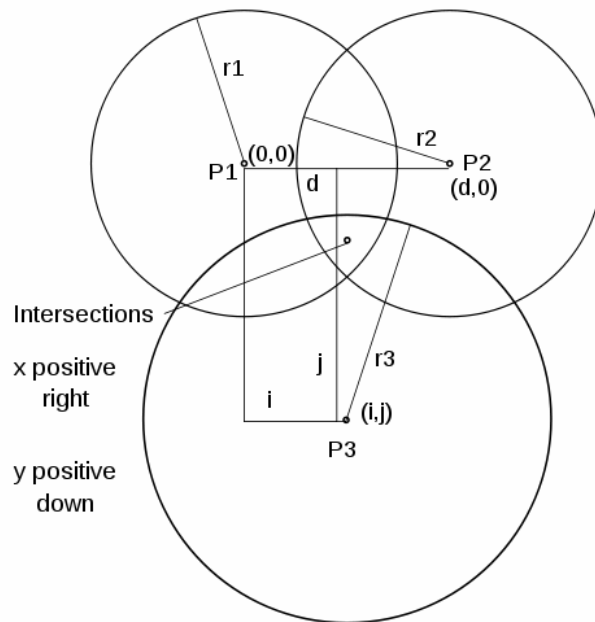
1. Satélites (GPS/GLONASS)

O hardware se comunica diretamente com constelações de satélites no espaço. Oferece alta precisão (metros), mas consome bastante bateria e não funciona bem em locais fechados.

2. Triangulação e Wi-Fi

O sistema operacional analisa a força do sinal das antenas de celular próximas e roteadores Wi-Fi locais para estimar a posição de forma rápida, mesmo sem visão do céu.

Múltiplas Fontes de Dados



Embora o termo popular seja **triangulação** (baseado em ângulos), o processo técnico mais comum utilizado pelas redes sem fio é a **trilateração**, que calcula a localização exata cruzando as **distâncias estimadas** a partir de três ou mais antenas

Desafio do GPS

O módulo GPS dedicado é extremamente "faminto" por energia. Atualizações contínuas em tempo real podem esgotar a bateria rapidamente.

Portanto, você só deve solicitar o uso em tempo real quando o aplicativo realmente depender dessa funcionalidade, parando o serviço imediatamente quando não for mais necessário.



Privacidade e Permissões



iOS

Políticas estritas da Apple. Uso prolongado de GPS gera notificações persistentes. O usuário tem controle refinado, escolhendo entre precisão exata ou raio geográfico.



Android

Sistema modular que separa permissões entre primeiro plano (Foreground) e segundo plano (Background). Exige justificativa rigorosa para a loja de aplicativos.

Permissões no IOS: O info.plist



Arquivo Declarativo Mandatário: Todas as intenções do aplicativo devem ser declaradas nativamente no arquivo Info.plist do iOS.



When In Use: A chave `NSLocationWhenInUseUsageDescription` é obrigatória para usar o mapa enquanto o app está aberto.



Always: A chave `NSLocationAlwaysAndWhenInUseUsageDescription` é necessária se você for rastrear o usuário com o app minimizado.



Risco de Rejeição: As descrições nessas chaves não podem ser genéricas. Se você escrever apenas "Precisamos do GPS", a Apple rejeita seu app nas lojas.

Permissões no IOS: O info.plist

Precise Location (Exata)

Utiliza todo o poder do hardware para determinar a rua, quarteirão ou até a porta onde o usuário está. Fundamental para apps como Uber e Google Maps.

Approximate Location (Aproximada)

Garante a privacidade, retornando apenas uma área com raio de alguns quilômetros. Ideal para apps de clima ou notícias locais, onde a rua exata não importa.

Permissões no Android: O Manifesto



Configuração Estática: Declaradas previamente dentro do arquivo AndroidManifest.xml.



ACCESS_FINE_LOCATION: Permissão máxima para ler os dados diretos do receptor de GPS (alta precisão).



ACCESS_COARSE_LOCATION: Estimativa aproximada, baseada primariamente em torres de telefonia e sinais de Wi-Fi.



ACCESS_BACKGROUND_LOCATION: Obrigatório para rodar em segundo plano, exigindo submissão de formulário no console da Google Play.

Permissões no Android: O Manifest

A partir do Android 6.0

Não basta apenas declarar no AndroidManifest. É obrigatório invocar um popup (Runtime Permission) perguntando ativamente ao usuário se ele aceita ceder a localização.

Apenas Desta Vez

Nas versões mais recentes do Android, o usuário pode conceder a permissão temporariamente. Se o app fechar, a permissão é automaticamente revogada pelo SO.

Boas Praticas de UX



Anti-Padrão

Nunca dispare os alertas de permissão de GPS no exato milissegundo em que o usuário instala e abre o aplicativo. Isso gera rejeição.



Explique o Motivo

Mostre uma tela de introdução explicando o valor agregado: "Precisamos do GPS para mostrar estabelecimentos perto de você".



Resiliência

Se o usuário negar a localização, seu aplicativo não deve "quebrar". Oculte a função e permita que ele use o resto do app.

Renderizando o Mapa

Para driblar os custos do Google Maps ou Apple Maps nativos, usaremos os quadros do **OpenStreetMap**.



Renderizando o Mapa

Por padrão, a biblioteca **react-native-maps** tenta carregar o Google Maps no Android e o Apple Maps no iOS.

Quando você coloca a propriedade `mapType="none"` no componente do mapa, você está mandando o aplicativo **desligar** os mapas padrões do sistema operacional (Google e Apple).

Em seguida, ao usar o componente `<UrlTile>` passando a URL do OpenStreetMap, você "pinta" os blocos gratuitos de mapa por cima da tela vazia, independente se o celular é iPhone ou Android.

```
useEffect(() => {  
  (async () => {  
    let { status } = await Location.requestForegroundPermissionsAsync();  
    if (status !== 'granted') {  
      setMensagem('Permissão NEGADA! O app não tem acesso.');      return;  
    }  
    setMensagem('Permissão CONCEDIDA! Buscando satélites...');  })();  
})
```

```
//inicia rastreo
await Location.watchPositionAsync(
  {
    accuracy: Location.Accuracy.High, //precisao do gps
    TimeInterval: 2000, // atualiza a cada 2 segundos
    distanceInterval: 1, // ou a cada 1 metro
  },
  (novaPosicao) => {
    setLocalizacao(novaPosicao.coords); // salva a posição sempre que andar
  }
);
})();
}, []);
```

O coração do aplicativo. O hardware de geolocalização será mantido ligado continuamente.

Usaremos **Accuracy.High**, atualizando a cada **2000ms** ou **1 metro** de deslocamento físico.

Parâmetros

accuracy (Precisão)

Define quão detalhada será a busca. Highest usa o GPS máximo, enquanto Balanced foca em economia usando Wi-fi. (Isso afeta a bateria intensamente).

timeInterval / distanceInterval

Define a frequência que o app é atualizado. No código acima: A cada 2 segundos (2000ms) **OU** a cada 1 metro deslocado no mundo físico.



1. Build via Expo Go

Instalem as dependências, abram o Expo Go no celular pessoal e leiam o QR Code para conectar na mesma rede local.



2. Código Nativo

Implementem a estrutura do em tempo real



3. Teste Físico

Caminhem pela sala ou corredor. Os números na tela precisam alterar em tempo real respondendo aos sensores físicos.

Prática – Permissão + Renderização Mapa

```
npx create-expo-app nome-app --template blank
```

Expo Go (SDK 57)

```
npx expo install expo-location
```

```
npx expo install react-native-maps
```

```
npx expo start
```

```
npx expo install expo-media-library
```

```
npx expo install expo-camera
```

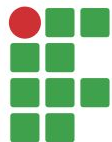
Dicas:

```
const [permissaoCamera, setPermissaoCamera] = useState(null);  
const [permissaoGaleria, setPermissaoGaleria] = useState(null);
```

```
// Criamos uma referência para "controlar" a câmera  
const cameraRef = useRef(null);
```

```
// Função que tira a foto e salva
```

milena.alegre@ifms.edu.br



INSTITUTO FEDERAL
Mato Grosso do Sul